

Cassandra CQL cheatsheet

1. DATABASE CREATION (KEYSPACE)

◆ Create Keyspace

```
1 CREATE KEYSPACE smart_factory
2 WITH replication = {
3     'class': 'SimpleStrategy',
4     'replication_factor': 3
5 };
```

Explanation

- `replication_factor = 3` → data stored on **3 nodes**
 - Ensures **fault tolerance + high availability**
-

◆ Use Keyspace

```
1 USE smart_factory;
```

◆ Delete Keyspace

```
1 DROP KEYSPACE smart_factory;
```

Test Cases

Case	Behavior
Create existing keyspace	✗ Error
Drop non-existing	✗ Error
Use before create	✗ Error



2. TABLE CREATION (CORE + ADVANCED)

◆ Basic Table

```
1 CREATE TABLE users (  
2     user_id UUID PRIMARY KEY,  
3     name TEXT,  
4     age INT,  
5     email TEXT  
6 );
```

◆ Composite + Clustering (REAL USE)

```
1 CREATE TABLE logs (  
2     user_id UUID,  
3     timestamp TIMESTAMP,  
4     event TEXT,  
5     PRIMARY KEY ((user_id), timestamp)  
6 ) WITH CLUSTERING ORDER BY (timestamp DESC);
```



Why This Design?

- Partition: `user_id` → all user logs together
 - Sorted by `timestamp` → fast recent queries
-

◆ Table Deletion

```
1 DROP TABLE users;
```

◆ Truncate (Delete All Rows)

```
1 TRUNCATE logs;
```



Test Cases

Case	Result
Insert without PK	❌ Error
Duplicate PK	✅ Overwrites
Wrong datatype	❌ Error

🔥 3. CRUD OPERATIONS (WITH EDGE CASES)

🟢 INSERT

```
1 INSERT INTO users (user_id, name, age)
2 VALUES (uuid(), 'Sagar', 22);
```

🧪 Edge Cases

Case	Result
Missing column	NULL
Same PK again	Overwrite
Large batch	⚠️ Slow

🟡 SELECT

```
1 SELECT * FROM users WHERE user_id = ?;
```

⚠️ Invalid Query

```
1 SELECT * FROM users WHERE age = 22;
```

👉 Unless indexed → ❌ error

UPDATE (UPSERT)

```
1 UPDATE users
2 SET age = 25
3 WHERE user_id = ?;
```

Edge Cases

Case	Result
Row not exist	Creates row
Update multiple cols	Allowed
No WHERE PK	✗ error

DELETE

```
1 DELETE FROM users WHERE user_id = ?;
```

Column Delete

```
1 DELETE age FROM users WHERE user_id = ?;
```

Edge Cases

Case	Result
Delete non-existing	No error
Delete column	Sets NULL
Delete with no PK	✗ error

4. COLLECTIONS (DETAILED + TEST CASES)

◆ LIST

```
1 subjects LIST<TEXT>
```

Insert

```
1 UPDATE students
2 SET subjects = ['math', 'cs']
3 WHERE id = ?;
```

Append

```
1 SET subjects = subjects + ['ai']
```

Remove

```
1 SET subjects = subjects - ['math']
```

◆ SET (UNIQUE)

```
1 subjects SET<TEXT>
```

```
1 SET subjects = subjects + {'ml'}
```

Test Case

Input	Output
Add duplicate	Ignored
Remove missing	No error

◆ MAP

```
1 marks MAP<TEXT, INT>
```

```
1 SET marks['math'] = 90;
```

Test Cases

Case	Result
Update key	Overwrites
Missing key	Added
Remove key	Deleted

COLLECTION PITFALLS

- Large collections →  performance issue
- Full rewrite may happen

5. TTL (EXPIRY)

Insert with TTL

```
1 INSERT INTO users (user_id, name)
2 VALUES (uuid(), 'temp')
3 USING TTL 60;
```

Check TTL

```
1 SELECT TTL(name) FROM users WHERE user_id = ?;
```

Remove TTL

```
1 UPDATE users USING TTL 0 SET name = 'permanent' WHERE user_id = ?;
```

Test Cases

Case	Result
TTL expires	Data gone
TTL reset	New timer
Too many TTL writes	Tombstone issue

6. COUNTERS

Create

```

1 CREATE TABLE page_views (
2     page TEXT PRIMARY KEY,
3     views COUNTER
4 );

```

Increment

```

1 UPDATE page_views
2 SET views = views + 1
3 WHERE page = 'home';

```

Test Cases

Case	Result
Insert	✗ Not allowed
Mix types	✗ Error
Negative update	✓ Allowed

7. AGGREGATION

Allowed

```
1 SELECT COUNT(*) FROM users;
2 SELECT MAX(age) FROM users;
```

Limited GROUP BY

```
1 SELECT user_id, COUNT(*) FROM logs GROUP BY user_id;
```

Pitfalls

- Full scan → slow
- Not for analytics

8. INDEXING

Create Index

```
1 CREATE INDEX ON users (age);
```

Use

```
1 SELECT * FROM users WHERE age = 22;
```

Test Cases

Case	Result
High cardinality	✗ slow
Low cardinality	✓ ok
Large table	✗ bad

Tip

 Prefer **data modeling over indexing**

9. CSV IMPORT / EXPORT

Import

```
1 COPY users (user_id, name, age)
2 FROM 'data.csv'
3 WITH HEADER = TRUE;
```

Export

```
1 COPY users TO 'output.csv'
2 WITH HEADER = TRUE;
```

Test Cases

Case	Result
Wrong datatype	✗ error
Missing column	✗ error
Large file	⚠ slow

Tips

- Use correct format
 - Use small batches
 - Ensure UUID format correct
-

10. IMPORTANT TIPS & TRICKS (HIGH VALUE)

1. Think Query First

Design table based on query, not normalization

2. Avoid ALLOW FILTERING

```
1 SELECT * FROM users WHERE age = 22 ALLOW FILTERING;
```

 BAD → full scan

3. Denormalization is GOOD

Duplicate data for speed

4. Partition Size

- Too big → slow
 - Too small → many partitions
-

5. Use Clustering Wisely

Sorting happens **only** inside partition

6. Avoid Large Collections

7. TTL Carefully

Too many → tombstones

8. No JOINS

Design tables accordingly



FINAL MASTER STATEMENT (WRITE IN EXAM)

Cassandra is a distributed, partitioned NoSQL database where schema design is query-driven. Operations like joins, aggregation, and offset are restricted to ensure scalability, high write throughput, and fault tolerance.



SORTING CHEAT SHEET (CQL)



Core

- Sorting = **clustering column only**
- Done at **storage level (not runtime)**



Table Definition

```
1 PRIMARY KEY ((user_id), timestamp)           -- default ASC
2 WITH CLUSTERING ORDER BY (timestamp DESC)    -- explicit DESC
```



Default Query

```
1 SELECT * FROM logs WHERE user_id = ?;
```

- Returns in **table order (ASC or DESC)**



ORDER BY

```
1 ORDER BY timestamp ASC | DESC
```

- Can **reverse order only**
- Works **only with partition key in WHERE**



Rules

- Only **clustering columns**
- Only **within ONE partition**
- No global sorting

- ❌ No non-key sorting
 - ❌ No OFFSET
-

Best Practice

- Use DESC for **latest-first queries**

```
1  LIMIT 10  -- fast recent data
```
